

Swahili Sentiment Analysis Report

1. Project Overview

This repository implements a Swahili sentiment analysis pipeline using both classical machine learning and transformer-based models. The main goals are:

- Compare classical TF-IDF based baselines against transformer fine-tuning.
- Evaluate Swahili sentiment performance on the AfriSenti dataset.
- Provide preprocessing, augmentation, training, and evaluation code.

The key models are:

- Logistic Regression (TF-IDF)
- Naive Bayes (Multinomial, TF-IDF)
- Linear SVM (TF-IDF)
- XLM-RoBERTa (fine-tuned)
- AfriBERTa (fine-tuned)

2. Repository Structure

Important directories

- ``data/raw/``
 - raw AfriSenti source files like ``dev.tsv``
- ``data/processed/``
 - generated cleaned dataset ``cleaned_data.csv``
 - optional augmented dataset ``augmented_data.csv``
- ``src/``
 - preprocessing, baseline models, transformers, augmentation, evaluation code
- ``notebooks/``
 - interactive preprocessing and baseline notebooks
- ``results/``
 - saved result CSVs, charts, and figure output

- ``models/``
- saved sklearn and transformer model artifacts
- ``tests/``
- unit tests for preprocessing and baseline training

Key files

- ``src/preprocess.py``
- ``src/baseline_models.py``
- ``src/transformer_models.py``
- ``src/augmentation.py``
- ``src/evaluate.py``
- ``requirements.txt``
- ``README.md``

3. Data and Labels

The processed dataset currently contains the following class counts in ``data/processed/cleaned_data.csv``:

- ``neutral``: 268 samples
- ``positive``: 137 samples
- ``negative``: 48 samples

This class imbalance is significant and means weighted metrics favour the neutral class more strongly.

Label encoding mapping in ``src/preprocess.py``:

- ``negative = 0``
- ``neutral = 1``
- ``positive = 2``

4. Baseline model pipeline

``src/baseline_models.py`` performs these steps:

- Load processed data from ``data/processed/cleaned_data.csv``
- Split data into train/test with stratification
- Vectorize tweets with TF-IDF
- Train:
 - Logistic Regression
 - Naive Bayes
 - Linear SVM
- Save:
 - ``results/baseline_results.csv``
 - ``models/`` pickle artifacts
 - confusion matrix charts

Metrics are computed using weighted averages for accuracy, F1, precision, and recall.

5. Transformer model pipeline

``src/transformer_models.py`` fine-tunes two transformer backbones:

- ``xlm-roberta-base``
- ``castorini/afribera_large``

It uses Hugging Face ``Trainer`` and evaluates on the held-out test split. The script currently:

- loads ``data/processed/cleaned_data.csv``
- splits train/test stratified by label
- tokenizes text
- fine-tunes each model
- saves:
 - ``results/transformer_results.csv``
 - ``results/figures/confusion_<model>.png``
 - model directories in ``models/``

If dependencies are missing, the script now prints a clear install message and exits.

6. Evaluation and reporting

`src/evaluate.py` merges baseline and transformer results and creates charts.

It expects:

- `results/baseline\_results.csv`
- `results/transformer\_results.csv` (optional)

It generates:

- `results/final\_comparison.csv`
- `results/figures/final\_comparison.png`
- `results/figures/f1\_ranked.png`
- `results/error\_analysis.txt`

7. Current stored results

From the current repository state, `results/transformer\_results.csv` contains:

- Model: XLM-RoBERTa
- Accuracy: 0.5934065934065934
- F1 Score (W): 0.4419856006062903
- Precision (W): 0.35213138509841807
- Recall (W): 0.5934065934065934

This indicates a low-weighted F1 score from the latest transformer run in this repo.

8. Issues found

Dependency issues

The main issue encountered during execution was missing transformer dependencies in the Python interpreter used to run the script.

The repository includes `torch`, `transformers`, and `sentencepiece` in `requirements.txt`, but the first runtime error showed:

- `ModuleNotFoundError: No module named 'torch'`
- later: `ModuleNotFoundError: No module named 'transformers'`

This was caused by the Python process not using the configured `.venv` environment.

Virtual environment status

A `.venv` virtual environment was created under the repo root, and `transformers` was installed there successfully.

The remaining problem is running the script with the correct `.venv` Python executable.

9. Recommendations

Best next steps

1. Activate the repo venv:

```
```powershell
cd c:\Users\katie\Downloads\swahili-sentiment\swahili-sentiment
.\.venv\Scripts\Activate.ps1
```
```

2. Run the transformer script with the venv Python:

```
```powershell
python src\transformer_models.py --model afriberta --epochs 5 --batch 32 --lr 1e-5
```
```

3. Then evaluate the combined results:

```
```powershell
python src\evaluate.py
```

...

### ***Improving F1***

- Use `AfriBERTa` rather than `XLM-RoBERTa`
- Increase training epochs
- Experiment with lower learning-rate and larger batch size
- Consider class balancing or data augmentation for minority classes

## **10. Summary**

This repository is functionally complete for Swahili sentiment analysis, with code for preprocessing, baseline training, transformer fine-tuning, augmentation, and evaluation.

The current repo state shows that the transformer portion has run with a poor F1 score, and execution errors were driven by environment mismatch rather than code structure.

The PDF generated from this report will capture all of the above findings in one document.